

TP — Audit & cartographie des données

Immobilier × Énergie — Loire-Atlantique (44)

Louis Valette

Alexis Fouquet

24 septembre 2026

Contents

1. Présentation du sujet	2
Contexte	2
Problématique	2
Objectif	2
Sujet retenu	2
Périmètre	2
Sommaire de la documentation	3
2. Sources de données	4
1. DVF — Demandes de valeurs foncières (géolocalisée)	4
2. DPE Logements existants (depuis juillet 2021)	4
3. Code Officiel Géographique (COG) — communes	5
Remarque sur data.gouv.fr	5
3. Dictionnaire de données	6
DVF — transactions immobilières	6
DPE — diagnostics de performance énergétique	6
COG — communes (référentiel géographique)	7
Clé de rapprochement entre les sources	7
4. Modèle conceptuel (MCD)	8
Entités et attributs	8
Relations et cardinalités	8
Justification des choix	9
Diagramme (dbdiagram.io)	9
5. Modèle logique (MLD)	10
commune	10
transaction_dvf	10
diagnostic_dpe	10
Relations (clés étrangères)	11
Index	11
6. Base PostgreSQL	12
Scripts	12

Comment reproduire	12
Résultats de la vérification (exécutée le 24/09/2026)	12
Résultat des requêtes d'analyse	12
7. Cartographie globale	14
Lecture du schéma	14
Annexe — Audit de la qualité des données	16
Résultats par source	16
Anomalie découverte lors des tests SQL	16
Valeur aberrante corrigée : biais des mutations DVF multi-lots	17
Vérification de bout en bout	17

1. Présentation du sujet

Contexte

La rénovation énergétique et la valeur des biens immobiliers sont deux enjeux étroitement liés en France : depuis 2021, le Diagnostic de Performance Énergétique (DPE) est opposable et influence de plus en plus la décision d'achat, tandis que les données de transactions immobilières (DVF) permettent d'objectiver les prix du marché. Croiser ces deux sources à l'échelle d'un département permet de sortir du discours général et d'observer, sur des données réelles, le lien entre performance énergétique et marché immobilier local.

Problématique

Dans le département de la **Loire-Atlantique (44)**, existe-t-il un écart de prix observable entre les logements performants (étiquettes DPE A/B/C) et les logements énergivores (étiquettes F/G), et comment cet écart varie-t-il selon les communes ?

Objectif

Construire, à partir de trois jeux de données publics réels, une base de données PostgreSQL structurée permettant :

- de rapprocher transactions immobilières et diagnostics énergétiques à l'échelle communale,
- de cartographier les communes du département selon le niveau de prix et la performance énergétique du parc de logements,
- de servir de support à des requêtes d'analyse (prix moyen au m² par étiquette DPE, par commune, etc.).

Sujet retenu

Immobilier × Énergie — croisement de deux sujets de la liste plutôt qu'un seul, pour donner du sens à l'analyse (un jeu de données immobilier seul ne dit rien sur l'énergie, et inversement).

Périmètre

Département : Loire-Atlantique (44), 207 communes. Choix motivé par un équilibre entre pôle urbain dense (Nantes, Saint-Nazaire) et zones rurales/littorales, offrant une diversité de types de logements et de niveaux de prix, avec une volumétrie de données maîtrisable pour un TP (dizaines à centaines de milliers de lignes, pas de traitement à l'échelle nationale).

Sommaire de la documentation

1. 01_presentation_sujet.md — ce document
2. 02_sources.md — sources de données
3. 03_dictionnaire_donnees.md — dictionnaire de données
4. 04_modele_conceptuel.md — modèle conceptuel (MCD)
5. 05_modele_logique.md — modèle logique (MLD)
6. 06_base_postgresql.md — base PostgreSQL
7. 07_cartographie_globale.md — cartographie globale du cheminement
8. annexe_audit_qualite.md — audit de qualité des données (travail préparatoire)

2. Sources de données

Trois sources réelles, toutes restreintes au département de la Loire-Atlantique (44). Pour chacune : nom, organisation/origine, URL, format, nature, description, et méthode de récupération effectivement utilisée.

1. DVF — Demandes de valeurs foncières (géolocalisée)

Champ	Valeur
Nom de la source	DVF géolocalisée (geo-dvf)
Organisation / origine	DGFIP, republiée/enrichie par Etalab
URL fiche	https://www.data.gouv.fr/datasets/demande-de-valeurs-foncieres-geolocalisee-1
URL fichier utilisé (dépt. 44)	https://files.data.gouv.fr/geo-dvf/latest/csv/2025/departements/44.csv.gz
Format	CSV compressé (.csv.gz)
Nature	Structurée (fichier tabulaire, une ligne = une disposition de mutation)
Description	Transactions immobilières réelles (mutations foncières : ventes de maisons, appartements, terrains, locaux) sur le département 44, géolocalisées, historique glissant. Chaque mutation peut être répartie sur plusieurs lignes (plusieurs lots/parcelles).
Licence	Licence Ouverte / Etalab
Récupération	Téléchargement direct du CSV compressé du département (curl), conservé tel quel dans <code>data/dvf_44_2025.csv.gz</code> (non versionné, cf. <code>.gitignore</code>)

Le dépôt GitHub [datagouv/dvf](https://github.com/datagouv/dvf) documente le schéma des colonnes.

2. DPE Logements existants (depuis juillet 2021)

Champ	Valeur
Nom de la source	DPE Logements existants
Organisation / origine	ADEME — Observatoire DPE-Audit
URL fiche	https://data.ademe.fr/datasets/dpe03existant
URL API utilisée	https://data.ademe.fr/data-fair/api/v1/datasets/dpe03existant/lines
Format	JSON via API (data-fair), converti en CSV
Nature	Structurée (schéma de 230 champs, un enregistrement = un diagnostic)
Description	Diagnostics de performance énergétique individuels (par logement) : étiquette énergie, étiquette GES, année de construction, surface habitable, type de bâtiment, adresse.
Licence	Licence Ouverte / Etalab
Récupération	Filtrage via le paramètre <code>qs=code_departement_ban:44</code> de l'API, pagination par curseur (next), export d'une sélection de 11 colonnes pertinentes vers <code>data/dpe_44.csv</code> (non versionné). ~352 000 diagnostics sur le département.

Champ de filtrage département confirmé dans le schéma de l'API : `code_departement_ban`. Champ de jointure commune : `code_insee_ban`.

3. Code Officiel Géographique (COG) — communes

Champ	Valeur
Nom de la source	Liste des communes de France (COG)
Organisation / origine	INSEE, republiée via data.gouv.fr
URL	https://www.data.gouv.fr/datasets/communes-et-villes-de-france-en-csv-excel-json-parquet-et-feather
Format	CSV
Nature	Structurée (une ligne = une commune)
Description	Table de référence géographique : code INSEE, nom, codes postaux, département, région, EPCI, population, superficie.
Licence	Licence Ouverte / Etalab
Récupération	Téléchargement du fichier national puis filtrage sur <code>dep_code = 44</code> (207 communes), écrit dans <code>data/cog_communes_44.csv</code> (non versionné)

Remarque sur data.gouv.fr

Conservé pour le COG (fiche officielle et stable), mais pour DVF et DPE les données sont récupérées directement depuis les portails sources qui font autorité (respectivement `files.data.gouv.fr/geo-dvf` et `data.ademe.fr`), plus fiables et mieux documentés que les entrées data.gouv.fr qui les référencent.

3. Dictionnaire de données

Analyse des principales colonnes retenues pour chaque source, restreintes au département 44. Seules les colonnes utiles au modèle sont détaillées (les fichiers sources bruts contiennent davantage de colonnes — 38 pour DVF, 230 pour le DPE).

DVF — transactions immobilières

Champ	Description	Type	Exemple
id_mutation	Identifiant de la mutation (peut se répéter sur plusieurs lignes si plusieurs lots/parcelles)	varchar	2025-563764
date_mutation	Date de la transaction	date	2025-01-06
nature_mutation	Type de mutation	varchar	Vente
valeur_fonciere	Prix de la transaction en euros	numeric	175000
code_postal	Code postal du bien	char(5)	44520
code_commune	Code INSEE de la commune	char(5)	44099
nom_commune	Nom de la commune	varchar	Moisdon-la-Rivière
code_departement	Code département	char(2)	44
id_parcelle	Identifiant cadastral de la parcelle	varchar	44099000AC0006
code_type_local	Code du type de local	integer	1
type_local	Type de bien	varchar	Maison, Appartement
surface_reelle_bati	Surface bâtie en m ²	numeric	98
nombre_pieces_principales	Nombre de pièces	integer	5
surface_terrain	Surface du terrain en m ²	numeric	945
longitude / latitude	Coordonnées géographiques	numeric	-1.37258 / 47.624643

DPE — diagnostics de performance énergétique

Champ	Description	Type	Exemple
numero_dpe	Identifiant unique du diagnostic	varchar	2644E0039039K
code_insee_ban	Code INSEE de la commune (géocodage BAN)	char(5)	44109
code_postal_ban	Code postal (géocodage BAN)	char(5)	44300
date_etablissement_dpe	Date de réalisation du diagnostic	date	2026-01-07
etiquette_dpe	Étiquette énergie (A à G)	char(1)	C
etiquette_ges	Étiquette gaz à effet de serre (A à G)	char(1)	C
annee_construction	Année de construction du bâtiment	integer	2020
periode_construction	Tranche de période de construction	varchar	2013-2021
type_batiment	Type de bâtiment	varchar	appartement, maison

Champ	Description	Type	Exemple
surface_habitable_logement	Surface habitable en m ²	numeric	64.3
adresse_ban	Adresse complète normalisée (BAN)	varchar	20 Route de Carquefou 44300 Nantes

COG — communes (référentiel géographique)

Champ	Description	Type	Exemple
code_insee	Code INSEE de la commune (clé primaire)	char(5)	44109
nom_standard	Nom usuel de la commune	varchar	Nantes
dep_code	Code département	char(2)	44
dep_nom	Nom du département	varchar	Loire-Atlantique
reg_code	Code région	char(2)	52
reg_nom	Nom de la région	varchar	Pays de la Loire
code_postal	Code postal principal	char(5)	44000
epci_code / epci_nom	Identifiant / nom de l'intercommunalité	varchar	244400404 / Nantes Métropole

Clé de rapprochement entre les sources

Les trois sources partagent un **code INSEE de commune à 5 caractères** :

- DVF : code_commune
- DPE : code_insee_ban
- COG : code_insee

C'est la clé utilisée pour la jointure géographique dans le modèle (cf. 04_modele_conceptuel.md).

4. Modèle conceptuel (MCD)

Entités et attributs

COMMUNE

Table de référence géographique, issue du COG, restreinte au département 44 (207 communes).

- code_insee (identifiant)
- nom_commune
- code_postal
- epci_code, epci_nom
- dep_code, dep_nom
- reg_code, reg_nom

TRANSACTION

Une transaction immobilière (mutation DVF), rattachée à une commune.

- id_transaction (identifiant)
- id_mutation
- date_mutation
- nature_mutation
- valeur_fonciere
- type_local
- surface_reelle_bati
- nombre_pieces_principales
- surface_terrain
- longitude, latitude

DIAGNOSTIC_DPE

Un diagnostic de performance énergétique, rattaché à une commune.

- numero_dpe (identifiant)
- date_etablissement_dpe
- etiquette_dpe
- etiquette_ges
- annee_construction
- periode_construction
- type_batiment
- surface_habitable_logement
- adresse

Relations et cardinalités

COMMUNE 1 — N TRANSACTION

COMMUNE 1 — N DIAGNOSTIC_DPE

- Une **commune** est concernée par **0 à N transactions** ; une **transaction** appartient à **exactement 1 commune**.
- Une **commune** est concernée par **0 à N diagnostics DPE** ; un **diagnostic** porte sur **exactement 1 commune**.

Justification des choix

Pourquoi ne pas relier directement TRANSACTION et DIAGNOSTIC_DPE ? Les deux sources ne partagent pas de clé fiable au niveau du logement individuel : DVF identifie une parcelle cadastrale (`id_parcelle`), le DPE une adresse géocodée (`adresse_ban`). Un rapprochement adresse-à-adresse est possible mais nécessite un géocodage/appariement flou (fuzzy matching), hors du périmètre d'un MCD simple. Le rapprochement est donc fait **au niveau commune**, la seule clé strictement commune et fiable aux deux sources (`code_insee`), ce qui suffit pour répondre à la problématique (comparer prix et performance énergétique par commune).

Pourquoi une entité COMMUNE séparée plutôt que dupliquer les infos géographiques dans chaque table ? Pour éviter la redondance (nom de commune, département, région répétés sur des dizaines de milliers de lignes) et garantir la cohérence : une seule source de vérité pour le référentiel géographique (COG), utilisée en contrainte d'intégrité référentielle sur les deux autres tables.

Diagramme (dbdiagram.io)

Schéma DBML prêt à coller sur dbdiagram.io :

```
Table commune {
  code_insee char(5) [pk]
  nom_commune varchar(100)
  code_postal char(5)
  epci_code varchar(15)
  epci_nom varchar(150)
  dep_code char(2)
  dep_nom varchar(50)
  reg_code char(2)
  reg_nom varchar(50)
}

Table transaction_dvf {
  id_transaction bigint [pk, increment]
  id_mutation varchar(20)
  date_mutation date
  nature_mutation varchar(50)
  valeur_fonciere numeric(12,2)
  code_insee char(5) [ref: > commune.code_insee]
  type_local varchar(50)
  surface_reelle_bati numeric(8,2)
  nombre_pieces_principales smallint
  surface_terrain numeric(10,2)
  longitude numeric(9,6)
  latitude numeric(9,6)
}

Table diagnostic_dpe {
  numero_dpe varchar(20) [pk]
  code_insee char(5) [ref: > commune.code_insee]
  date_etablissement_dpe date
  etiquette_dpe char(1)
  etiquette_ges char(1)
  annee_construction integer
  periode_construction varchar(20)
  type_batiment varchar(30)
  surface_habitable_logement numeric(8,2)
  adresse varchar(255)
}
```

5. Modèle logique (MLD)

Traduction du modèle conceptuel (04_modele_conceptuel.md) en tables relationnelles. Script de création correspondant : [sql/01_create_tables.sql](#).

commune

Colonne	Type	Contrainte
code_insee	CHAR(5)	PK
nom_commune	VARCHAR(100)	NOT NULL
code_postal	CHAR(5)	
epci_code	VARCHAR(15)	
epci_nom	VARCHAR(150)	
dep_code	CHAR(2)	NOT NULL
dep_nom	VARCHAR(50)	
reg_code	CHAR(2)	
reg_nom	VARCHAR(50)	

transaction_dvf

Colonne	Type	Contrainte
id_transaction	BIGSERIAL	PK
id_mutation	VARCHAR(20)	NOT NULL
date_mutation	DATE	NOT NULL
nature_mutation	VARCHAR(50)	
valeur_fonciere	NUMERIC(12,2)	NOT NULL, CHECK ≥ 0
code_insee	CHAR(5)	FK $\rightarrow$ commune(code_insee), NOT NULL
type_local	VARCHAR(50)	
surface_reelle_bati	NUMERIC(8,2)	CHECK ≥ 0
nombre_pieces_principales	SMALLINT	
surface_terrain	NUMERIC(10,2)	CHECK ≥ 0
longitude	NUMERIC(9,6)	
latitude	NUMERIC(9,6)	

Notation : transaction_dvf porte une clé primaire technique (id_transaction) car une même mutation DVF (id_mutation) peut apparaître sur plusieurs lignes du fichier source (un lot/parcelle par ligne) — id_mutation est donc conservé comme attribut, pas comme clé.

diagnostic_dpe

Colonne	Type	Contrainte
numero_dpe	VARCHAR(20)	PK
code_insee	CHAR(5)	FK $\rightarrow$ commune(code_insee), NOT NULL
date_etablissement_dpe	DATE	
etiquette_dpe	CHAR(1)	CHECK IN (A..G)
etiquette_ges	CHAR(1)	CHECK IN (A..G)
annee_construction	INTEGER	
periode_construction	VARCHAR(20)	
type_batiment	VARCHAR(30)	

Colonne	Type	Contrainte
surface_habitable_logement	NUMERIC(8,2)	CHECK ≥ 0
adresse	VARCHAR(255)	

Relations (clés étrangères)

- transaction_dvf.code_insee $\rightarrow$ commune.code_insee
- diagnostic_dpe.code_insee $\rightarrow$ commune.code_insee

Index

- commune : PK sur code_insee (index automatique)
- transaction_dvf : index sur code_insee (jointures), index sur date_mutation
- diagnostic_dpe : index sur code_insee (jointures), index sur etiquette_dpe

6. Base PostgreSQL

Scripts

Script	Rôle
sql/01_create_tables.sql	Création des 3 tables, contraintes, index
sql/02_insert_test_data.sql	Quelques données de test manuelles (3 communes, 3 transactions, 3 diagnostics)
sql/03_import_donnees_reelles.sql	Import des données réelles nettoyées du département 44 (data/staging/)
sql/04_requetes_verification.sql	Contrôle des relations (comptages, orphelins)
sql/05_requetes_analyse.sql	Requêtes répondant à la problématique du TP

Comment reproduire

Option A — via Docker (utilisée pour valider les scripts)

```
docker run -d --name tp_audit_44_pg -e POSTGRES_PASSWORD=postgres \
-e POSTGRES_DB=audit_immo_energie_44 -p 5433:5432 postgres:16

psql -h localhost -p 5433 -U postgres -d audit_immo_energie_44 -f sql/01_create_tables.sql
psql -h localhost -p 5433 -U postgres -d audit_immo_energie_44 -f sql/02_insert_test_data.sql # données de test
# ou, pour les données réelles (depuis la racine du projet, où se trouve data/staging/) :
psql -h localhost -p 5433 -U postgres -d audit_immo_energie_44 -f sql/03_import_donnees_reelles.sql
```

Option B — via pgAdmin

1. Créer une base `audit_immo_energie_44`.
2. Ouvrir le Query Tool, exécuter `01_create_tables.sql`.
3. Exécuter `02_insert_test_data.sql` pour les données de test, ou importer les CSV de `data/staging/` via l'outil d'import graphique de pgAdmin pour les données réelles.

Résultats de la vérification (exécutée le 24/09/2026)

Base créée et testée sur une instance PostgreSQL 16 locale (Docker), avec import complet des trois sources nettoyées (cf. annexe_audit_qualite.md) :

Table	Lignes importées
commune	207
transaction_dvf	79 315
diagnostic_dpe	351 959

- **0 transaction orpheline, 0 diagnostic orphelin** : toutes les clés étrangères vers commune sont valides.
- Les contraintes CHECK (étiquettes DPE/GES dans A–G, surfaces et prix ≥ 0) sont passées sans erreur sur l'ensemble du jeu réel.

Résultat des requêtes d'analyse

Le prix moyen au m² est calculé au niveau de la mutation (et non de la ligne brute) pour éviter un biais des mutations DVF à plusieurs lots — cf. annexe_audit_qualite.md pour le détail du problème rencontré et de la correction apportée.

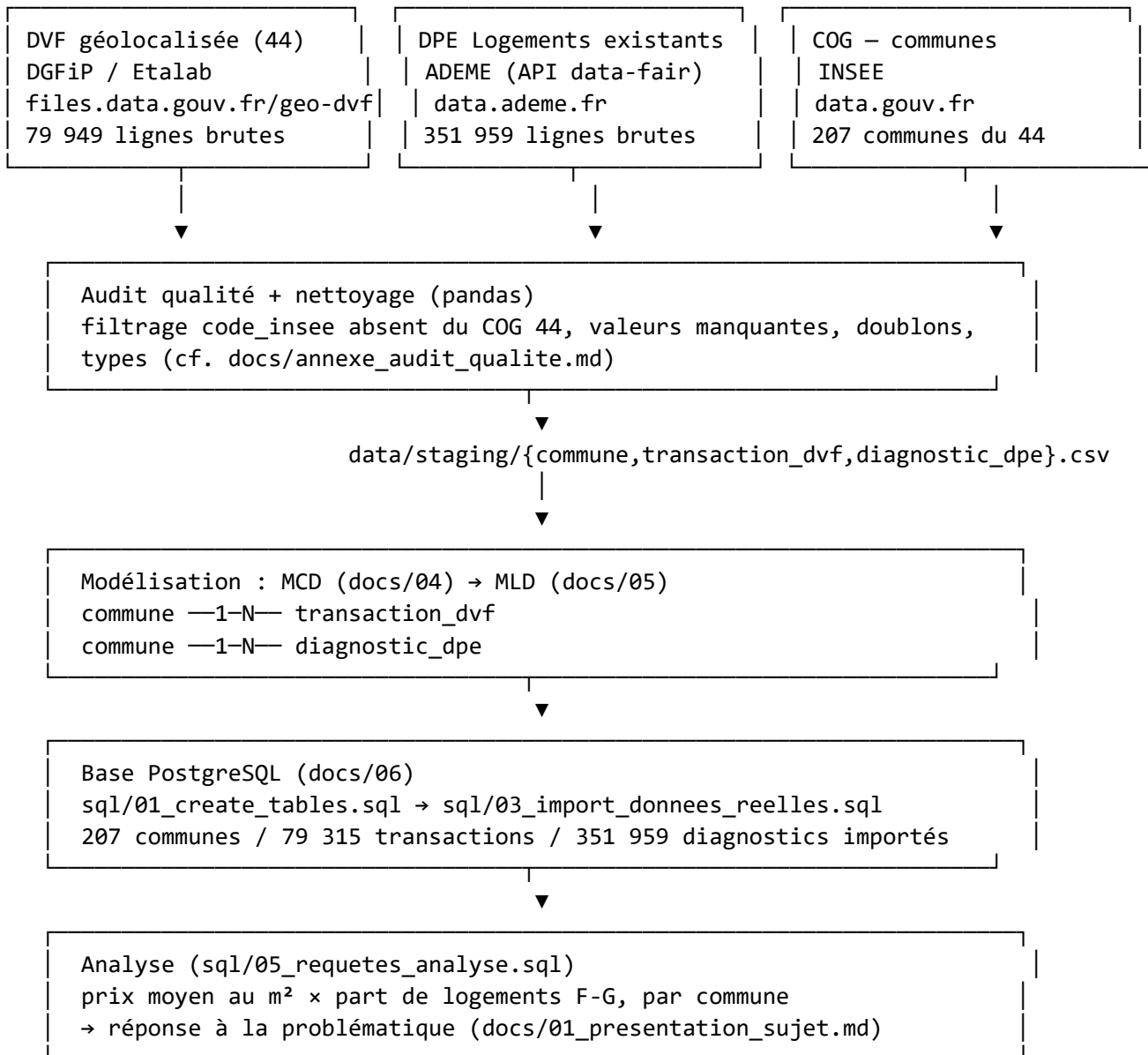
Sur les données réelles corrigées, le croisement prix moyen au m² / part de logements F-G par commune (sql/05_requetes_analyse.sql, requête 3) fait apparaître deux tendances :

- Les communes les plus chères du département sont ses pôles littoraux (La Baule-Escoublac 6 536 €/m², Pornichet 5 647 €/m², Le Pouliguen 5 290 €/m², Pornic 4 840 €/m²), suivies de Nantes (3 778 €/m², rang médian malgré son statut de préfecture) et de sa périphérie (Saint-Herblain 2 793 €/m², Orvault 3 177 €/m²).
- Les communes rurales les moins chères concentrent la plus forte part de logements F-G : Juigné-des-Moutiers (892 €/m², 35,7 % de F-G), La Chapelle-Glain (921 €/m², 19,4 %), Soulvache (1 015 €/m², 25,7 %), Grand-Auverné (1 102 €/m², 20,4 %).

Cette observation va dans le sens de la problématique posée (01_presentation_sujet.md) mais reste à nuancer : les communes rurales les moins chères ont de faibles effectifs (souvent 10 à 20 mutations), et un parc ancien plus présent en zone rurale peut expliquer à la fois le prix bas et la mauvaise étiquette, sans lien de causalité direct entre les deux. Le prix des communes littorales, à l'inverse, reflète surtout la rareté foncière (proximité de la mer) plus que la performance énergétique du bâti.

7. Cartographie globale

Vue d'ensemble du cheminement, des données brutes à la base PostgreSQL.



Lecture du schéma

1. **Trois sources hétérogènes** (formats et organisations différentes) sont ramenées à un périmètre commun : le département 44.
2. **L'audit qualité** filtre et nettoie chaque source indépendamment avant toute jointure, en s'appuyant sur une clé géographique commune fiable (code_insee).
3. **La modélisation** choisit délibérément une architecture "en étoile" simple (deux tables de faits reliées uniquement par la dimension commune) plutôt qu'un rapprochement fin transaction ↔ diagnostic, jugé trop incertain avec les clés disponibles (cf. justification dans 04_modele_conceptuel.md).
4. **La base PostgreSQL** matérialise ce modèle et a été testée de bout en bout avec les données réelles (0 ligne orpheline).

5. **L'analyse finale** répond à la problématique initiale à l'échelle communale, après correction d'un biais de calcul sur les mutations DVF multi-lots (cf. annexe_audit_qualite.md), avec ses limites restantes documentées (petits effectifs sur les communes rurales, corrélation $\neq$ causalité).

Annexe — Audit de la qualité des données

Travail préparatoire (pas un livrable en soi) mené avant de figer le dictionnaire de données et le modèle. Réalisé avec un script Python (pandas) qui nettoie les trois extraits bruts et produit les fichiers prêts à l'import dans data/staging/.

Résultats par source

COG (référentiel communes)

- 207 lignes brutes, 207 après nettoyage.
- Aucun doublon de code_insee, aucune valeur manquante sur la clé.
- Confirme le nombre officiel de communes de Loire-Atlantique (207).

DVF (transactions)

- 79 949 lignes brutes → **79 315 lignes retenues**.
- 634 lignes sans valeur_fonciere exploitable, retirées (transactions sans prix renseigné, ex. certaines expropriations).
- 53 205 lignes sans surface_reelle_bati : normal, une partie des mutations DVF concerne des terrains non bâtis (pas une anomalie, mais un point d'attention pour les requêtes d'analyse — cf. sql/05_requetes_analyse.sql qui filtre sur type_local IN ('Maison', 'Appartement') et surface_reelle_bati > 0).
- Répartition de nature_mutation : très majoritairement des ventes classiques (75 243 sur 79 949), le reste = ventes en VEFA, échanges, adjudications.
- Répartition de type_local : beaucoup de lignes “Dépendance” (garages, caves — 21 271) et de lignes sans type (terrains, 31 825), à filtrer selon la question posée.
- 0 ligne avec un code_insee hors du référentiel COG 44 — bon signe de cohérence entre les deux sources.

DPE (diagnostics)

- 351 959 lignes brutes → **351 959 lignes retenues** (aucune ligne perdue).
- 0 doublon sur numero_dpe.
- Étiquettes DPE et GES 100 % dans l'ensemble attendu A–G, aucune valeur aberrante.
- 0 ligne avec un code_insee_ban hors du référentiel COG 44.
- **Anomalie détectée** : 126 lignes (0,04 %) présentent un artefact d'encodage dans le champ adresse — un caractère accentué mal ré-encodé apparaît littéralement comme une séquence du type *u00ee au lieu du caractère attendu (ex. î). L'anomalie est présente dès la réponse brute de l'API ADEME, donc en amont de notre traitement — c'est un défaut de qualité de la source, pas de notre pipeline. Volume négligeable, non corrigé (pas bloquant pour l'exploitation en base), mais documenté ici pour traçabilité.

Anomalie découverte lors des tests SQL

En testant la requête de vérification croisant transaction_dvf et diagnostic_dpe par commune (sql/04_requetes_verification.sql), une première version jointait les deux tables directement sur commune dans la même requête. Sur une commune comme Nantes (14 195 transactions × 132 678 diagnostics), cela produit un **produit cartésien** de près de 2 milliards de lignes avant agrégation — requête bloquée en pratique. Corrigé en remplaçant la double jointure par deux sous-requêtes indépendantes (SELECT COUNT(*) FROM ... WHERE code_insee = c.code_insee), beaucoup plus efficace et correcte. Bon exemple concret de pourquoi le modèle “étoile” (deux faits reliés uniquement par la dimension commune, jamais entre eux) doit être interrogé prudemment.

Valeur aberrante corrigée : biais des mutations DVF multi-lots

La première version de la requête de prix moyen au m² par commune (`sql/05_requetes_analyse.sql`, requête 1) faisait ressortir Saint-Philbert-de-Grand-Lieu à 26 644 €/m², très au-dessus de toutes les autres communes (deuxième plus chère à l'époque : Machecoul-Saint-Même à 15 360 €/m², Nantes à 8 677 €/m²).

Cause identifiée : dans DVF, une mutation portant sur plusieurs lots (ex. un lotissement de plusieurs maisons vendu par un promoteur en une seule transaction) est éclatée sur plusieurs lignes du fichier source — une ligne par lot — mais `valeur_fonciere` (le prix total de la mutation) est **répété à l'identique sur chaque ligne**, pas réparti. La requête initiale divisait cette valeur totale par la surface de chaque ligne individuellement, ce qui démultiplie artificiellement le prix au m² dès qu'une mutation comporte plusieurs lots.

Exemple concret trouvé sur Saint-Philbert-de-Grand-Lieu : la mutation 2025-583422 (5 591 000 €, vente d'un lotissement) apparaît sur 82 lignes du fichier département 44, chacune avec la même `valeur_fonciere` de 5 591 000 €. Le calcul naïf traitait donc 82 fois cette même vente comme si chaque maison valait individuellement 5,6 M€. Ce n'est pas un cas isolé : sur les 31 633 mutations distinctes du département, 1 877 comportent plus de 5 lignes — le biais affecte donc l'ensemble du classement, pas seulement la commune la plus visible.

Correction appliquée : la requête agrège désormais d'abord par `id_mutation` (une valeur foncière par mutation, somme des surfaces des lots Maison/Appartement de cette mutation), puis calcule le prix au m² sur cette base, avant de moyenner par commune. Résultat : Saint-Philbert-de-Grand-Lieu retombe à 2 347 €/m² (rang médian, cohérent avec ses voisines), et le classement général devient dominé par les communes littorales (La Baule-Escoublac 6 536 €/m², Pornichet 5 647 €/m², Le Pouliguen 5 290 €/m²), un résultat nettement plus plausible pour le marché immobilier du 44. Voir `06_base_postgresql.md` pour les chiffres à jour.

Vérification de bout en bout

Les scripts SQL ont été testés sur une instance PostgreSQL 16 locale (conteneur Docker), avec :

- création des tables (`01_create_tables.sql`) : OK
- insertion des données de test (`02_insert_test_data.sql`) : OK, relations vérifiées (0 ligne orpheline)
- import complet des données réelles nettoyées (`03_import_donnees_reelles.sql`) : OK — 207 communes, 79 315 transactions, 351 959 diagnostics, **0 ligne orpheline**
- requêtes d'analyse (`05_requetes_analyse.sql`) : OK — résultats cohérents avec la problématique (cf. `06_base_postgresql.md`)